

```

        message.send()
    else:
        greeting = ('<a href="%s">Sign
in or Register</a>.' % users.create_login_url('/'))
        self.response.out.
write('<html><body>%s</body></html>' % greeting)
    app = webapp2.WSGIApplication([
        ('/', MainHandler),
        ('/signup', SignUp),
    ], debug=True)

```

If you're new to python, you should understand that indentation is pivotal in the language and copy-pasting will not do you much good in most cases.

8. Save your work.
9. To locally test your application. Click on Run, then Browse. All errors can be found in Logs.
10. Deploy your app using App Engine Launcher using your google account and close when prompted.
11. Your application can now be found at your-app-id@appspot.com if no error occurs.
12. At appengine.google.com, you can find your dashboard where the Quota limitations for your free application can be checked and upgrade to a paid account can be made as well for more traffic purposes.

Code Explanation:

Everything there is to know about App Engine using Python can be found at <http://dgit.in/py4dumis> but we'll be discussing only what's relevant to this workshop.

Webapp2 is a python framework used to make web applications. Two services have been used in the making of this project – users and mail. The mail service however cannot be tested locally and the users service does not authenticate locally and therefore any test e-mail id can be used. The class MainHandler is configured to execute at '/' i.e. the home page. It takes webapp2.RequestHandler as an argument. RequestHandler classes process and build responses. It has only one 'get' function that takes the current instance 'self' as the argument. While print command is used to execute to the console, self.response.out.write belongs to the response class and is a StringIO instance. Typically, the WSGIApplication instantiates a RequestHandler and initializes it with a Response object with default values. After which, the contents of this object are sent as the body of the response when the request handler method returns. The contents of the method is a form element that submits on clicking Sign Up and redirects to '/signup'. The triple quotes (""") are used to write on multiple lines. The .out can be avoided. The class SignUp is configured to execute at '/signup' by the WSGIApplication method. It also takes in the same parameter as MainHandler and has only 'post' function that takes 'self' as argument. 'Hey, there!' is set as a default greeting to check whether the function is running correctly and that the problem probably lies in the subsequent lines of code. This is a common debugging technique. The users class has a method called get_current_user() which is stored in the variable user. If the user variable is not null, then a welcome message is stored in the 'greeting' variable along with options of Sign Out and Not You? and a mail is sent to the current user. The %s are replaced in order with the variables at the end enclosed as %(-,-,-). The string replacement in this case is done using the other functions of user class viz.

nickname(), create_logout_url() and create_login_url() which can be read about at <http://dgit.in/ldr1axT> and <http://dgit.in/1aGiUly> The next lines of code are written to configure the outgoing e-mail that is sent to the user. There are other alternatives to this method but we prefer this for its clarity. An EmailMessage instance represents an email message to be sent using the App Engine Mail service. This instance is stored in message. The following lines of code initialize the sender, subject, to and body fields for the constructor. The mail is finally sent using the send(). More about the functions and fields of mail class can be found at <http://dgit.in/1aOpbe1> and <http://dgit.in/17XfuXc>

If the user variable is null, then the link for Sign in or Register is stored in the greeting variable.

According to the case, the greeting variable is displayed on the screen. A string type variable can have many associated function that can be learnt about from <http://docs.python.org/2/library/string.html> These lines of code configure the webapp2 by using the WSGIApplication instance that routes incoming requests to handlers based on the URL

In this workshop, we have not edited the app.yaml file for simplicity but there are a few things you should know about App Engine and the configuration file.

1. In the app.yaml file, make sure your application has the same identifier as the one given by you during creating your application at appengine.google.com
2. The version number allows the developer to rollback to a previous version after deploying a newer version with the modified app.yaml file.
3. The handler for the .ico url can be removed but for the url: /* the script must remain the same as the name of the file which contains the class configured to execute at the url. For ex. main.py contains MainHandler class that is configured to execute at '/' and thus the script for the url must be main.app
4. You can learn more about it using the reference provided at <http://dgit.in/1aOpfdF>
6. The web server knows to look for changes and executed these changes on refresh which means that while your app is running, you can continue to make changes to it without stopping it.

Possibilities

If you like the possibilities of creating applications using Google App Engine, your next steps should be to learn the back-end framework, datastore which runs on google's own querying language GQL and also learn how to use jinja templates and link static files like css to improve the presentation of your application on the web. For a tutorial by Reddit co-founder Steve Huffman

on the subject, visit <https://www.udacity.com/course/cs253> available free of cost. If you'd like to get a glimpse of the possibilities that can be accomplished using the App Engine visit this gallery of apps: <http://dgit.in/appitnow> or if you'd like to check out projects that were modified or extended using an App Engine application using an App Engine Patch check out: <http://dgit.in/appaspatch>

So at no cost, you get to develop 10 web applications or complete websites with a significantly small price of quotas which are quite large for a small-scale project like a blog. We hope to see further developments in the App Engine environment with support for additional functionality. Thank you for reading! And don't forget to make your application thread-safe. >